

Exception Handling in Java for Beginners

A practical guide to exception handling. We start with the basics and work our way up to the patterns you will see in interviews and real production code.

This is one comprehensive guide rather than three tiers, since the topic flows naturally from start to finish.

What is an Exception?

An exception is an event that breaks the normal flow of your program. It usually means something unexpected happened.

Common reasons:

- Trying to read a file that does not exist
- Dividing by zero
- Calling a method on a `null` object
- Accessing an array index that is out of bounds
- A network call timed out

When something like this happens, Java creates an exception object and "throws" it. If your code does not catch it, the program crashes.

A simple example

```
java
int[] nums = {1, 2, 3};
System.out.println(nums[10]); // crash!
```

Output:

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException:
Index 10 out of bounds for length 3
```

Your program stops right there. Anything after this line never runs.

Handling Exceptions with try-catch

To prevent the crash, wrap risky code in a `try` block. If something goes wrong, the `catch` block runs.

```
java

try {
    int[] nums = {1, 2, 3};
    System.out.println(nums[10]);
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Index out of bounds: " + e.getMessage());
}

System.out.println("Program continues normally");
```

Output:

```
Index out of bounds: Index 10 out of bounds for length 3
Program continues normally
```

The exception was caught, handled, and the program kept going.

What can you do with the exception object?

```
java

catch (Exception e) {
    e.getMessage();           // short description of what went wrong
    e.getCause();             // another exception that caused this one (or null)
    e.printStackTrace();      // print the full stack trace
}
```

The Exception Hierarchy

All exceptions in Java extend from a class called `Throwable`. The hierarchy looks like this:

```
Throwable
├── Error                                (serious JVM problems, do not catch)
│   ├── OutOfMemoryError
│   └── StackOverflowError
└── Exception                          (problems your code can handle)
    ├── IOException                    (checked)
    ├── SQLException                  (checked)
    └── RuntimeException                (unchecked)
        ├── NullPointerException
        ├── IllegalArgumentException
        ├── IndexOutOfBoundsException
        ├── ClassCastException
        └── ... many more
```

Error vs Exception

- **Error:** Something is wrong with the JVM itself. Out of memory, stack overflow, etc. You usually do not catch these. The program is in deep trouble.
- **Exception:** A problem your code can deal with. This is what you handle.

Checked vs Unchecked Exceptions

This is one of the most asked interview questions. Know it well.

- **Checked exceptions** extend `Exception` but NOT `RuntimeException`. The compiler forces you to handle them or declare them. Examples: `IOException`, `SQLException`.
- **Unchecked exceptions** extend `RuntimeException`. The compiler does NOT force you to handle them. Examples: `NullPointerException`, `IllegalArgumentException`.

java

```
// Checked: must handle or declare
public void readFile() {
    Files.readString(Path.of("file.txt"));    // does not compile!
}

// Either handle it:
public void readFile() {
    try {
        Files.readString(Path.of("file.txt"));
    } catch (IOException e) {
        // handle it
    }
}

// Or declare it:
public void readFile() throws IOException {
    Files.readString(Path.of("file.txt"));
}
```

```
java
```

```
// Unchecked: no compile error, but can crash at runtime
String s = null;
s.length();    // throws NullPointerException at runtime
```

A simple way to remember

If the exception name ends in `Exception` and is not under `RuntimeException`, it is checked. Most things in I/O and database code are checked.

throw : Creating an Exception

You can throw exceptions from your own code.

```
java

public void setAge(int age) {
    if (age < 0) {
        throw new IllegalArgumentException("Age cannot be negative");
    }
    this.age = age;
}
```

Use `throw` followed by `new SomeException(...)`. The method stops right there and Java starts looking for a matching `catch` block.

Throwing in the right places

```
java

public User findUser(int id) {
    if (id <= 0) {
        throw new IllegalArgumentException("id must be positive");
    }

    User user = database.lookup(id);
    if (user == null) {
        throw new UserNotFoundException("No user with id " + id);
    }

    return user;
}
```

Throwing exceptions is your way of saying "I cannot continue, something is wrong."

throws : Declaring an Exception

If your method might throw a checked exception, you must declare it with `throws` in the method signature.

```
java

public String readFile(String path) throws IOException {
    return Files.readString(Path.of(path));
}
```

Anyone who calls this method now has two choices.

Choice 1: Handle the exception

```
java

try {
    String content = readFile("data.txt");
    System.out.println(content);
} catch (IOException e) {
    System.err.println("Failed to read: " + e.getMessage());
}
```

Choice 2: Pass it along

```
java

public void processFile() throws IOException {
    String content = readFile("data.txt");
    // ...
}
```

This is called "ducking" the exception. Eventually someone has to handle it.

Multiple exceptions

```
java

public void doStuff() throws IOException, SQLException {
    // method body
}
```

The finally Block

Code in `finally` always runs. It runs after the try block. It runs even if there is an exception. It runs even if you return early.

```
java

try {
    System.out.println("Inside try");
    throw new RuntimeException("Oops");
} catch (RuntimeException e) {
    System.out.println("Caught: " + e.getMessage());
} finally {
    System.out.println("Always runs");
}
```

Output:

```
Inside try
Caught: Oops
Always runs
```

What is finally used for?

Mainly to clean up resources, like closing files or database connections.

```
java

FileReader reader = null;
try {
    reader = new FileReader("file.txt");
    // do stuff
} catch (IOException e) {
    System.err.println("Read failed");
} finally {
    if (reader != null) {
        try {
            reader.close();
        } catch (IOException e) {
            // ignore
        }
    }
}
```

This works, but it is ugly and easy to mess up. There is a much better way, which we cover next.

try-with-resources (Java 7+)

For anything that needs cleanup (files, connections, streams), use try-with-resources. Java closes them for you automatically.

The clean way

```
java

try (FileReader reader = new FileReader("file.txt")) {
    // do stuff
} catch (IOException e) {
    System.err.println("Read failed: " + e.getMessage());
}
```

That is it. No `finally`. Java calls `close()` on the reader when the block ends, even if there is an exception.

Multiple resources

```
java
```

```
try (  
    FileReader reader = new FileReader("input.txt");  
    FileWriter writer = new FileWriter("output.txt")  
) {  
    // read from reader, write to writer  
} catch (IOException e) {  
    System.err.println("Failed: " + e.getMessage());  
}
```

Both are closed automatically. They are closed in reverse order of declaration (writer first, then reader).

What works in try-with-resources?

Any class that implements `AutoCloseable` (or its sub-interface `Closeable`). That includes:

- `FileReader`, `FileWriter`
- `BufferedReader`, `BufferedWriter`
- `InputStream`, `OutputStream`
- `Connection`, `Statement`, `ResultSet` (JDBC)
- `Scanner`
- Most Java I/O classes

Make your own AutoCloseable

```
java
```

```
public class MyResource implements AutoCloseable {  
    public MyResource() {  
        System.out.println("Opened");  
    }  
  
    public void use() {  
        System.out.println("Using");  
    }  
  
    @Override  
    public void close() {  
        System.out.println("Closed");  
    }  
}  
  
try (var r = new MyResource()) {  
    r.use();  
}
```

Output:

```
Opened  
Using  
Closed
```

The `close()` method runs automatically.

Use try-with-resources whenever you can

If you are using a `finally` just to close something, you should be using try-with-resources instead.

Multiple catch Blocks

You can catch different exception types separately.

```
java
```

```
try {
    doRiskyStuff();
} catch (IOException e) {
    System.err.println("File problem");
} catch (SQLException e) {
    System.err.println("Database problem");
} catch (Exception e) {
    System.err.println("Something else: " + e.getMessage());
}
```

Order matters

More specific catches go first. If you put `Exception` first, it would catch everything and the others would never run.

```
java
// Wrong: this will not compile
try {
    ...
} catch (Exception e) {           // catches everything
    ...
} catch (IOException e) {         // unreachable!
    ...
}
```

The compiler catches this mistake.

Multi-catch (Java 7+)

If you handle multiple exception types the same way, combine them with `|`.

```
java
try {
    doRiskyStuff();
} catch (IOException | SQLException e) {
    System.err.println("IO or DB problem: " + e.getMessage());
    log.error("Operation failed", e);
}
```

Cleaner than two separate catch blocks with the same code.

Things to know

- The variable `e` is effectively final inside multi-catch. You cannot reassign it.

- You cannot combine exceptions where one is a parent of the other (like `IOException` | `Exception`). It would be redundant.
-

Common Built-in Exceptions

These come up everywhere. Know what each one means.

NullPointerException (NPE)

You called a method or accessed a field on a `null` reference.

```
java

String s = null;
s.length();    // NPE
```

IllegalArgumentException

A method got an argument it does not accept.

```
java

public void setAge(int age) {
    if (age < 0) {
        throw new IllegalArgumentException("Age must be non-negative");
    }
}
```

IllegalStateException

The object is in a state where this method cannot be called.

```
java

Iterator<Integer> it = list.iterator();
it.remove();    // IllegalStateException, you must call next() first
```

ArrayIndexOutOfBoundsException

You accessed an array index that does not exist.

```
java

int[] nums = {1, 2, 3};
int x = nums[5];    // crash
```

IndexOutOfBoundsException

Same idea but for `List`, `String`, etc.

```
java  
  
List<Integer> list = List.of(1, 2, 3);  
list.get(10);    // crash
```

ClassCastException

You tried to cast an object to a wrong type.

```
java  
  
Object o = "hello";  
Integer i = (Integer) o;    // crash
```

NumberFormatException

You tried to parse a string that is not a valid number.

```
java  
  
int n = Integer.parseInt("hello");    // crash
```

ConcurrentModificationException

You modified a collection while iterating it.

```
java  
  
for (Integer n : list) {  
    list.remove(n);    // crash  
}
```

Use the iterator's `remove()` method or `removeIf()` instead.

```
java  
  
list.removeIf(n -> n > 5);    // safe
```

ArithmeticException

Math errors like dividing by zero (with `int`, not `double`).

```
java
```

```
int x = 10 / 0;    // crash

double y = 10.0 / 0.0;    // not a crash, gives Infinity
```

IOException (checked)

File or stream operations failed.

SQLException (checked)

Database operations failed.

UnsupportedOperationException

You called a method that is not supported on this object.

```
java

List<Integer> nums = List.of(1, 2, 3);
nums.add(4);    // crash, immutable list
```

Custom Exceptions

Sometimes the built-in exceptions do not capture what is wrong. You can create your own.

Custom unchecked exception

Most custom exceptions should extend `RuntimeException`.

```
java

public class UserNotFoundException extends RuntimeException {
    public UserNotFoundException(String message) {
        super(message);
    }

    public UserNotFoundException(String message, Throwable cause) {
        super(message, cause);
    }
}
```

Using it:

```
java
```

```

public User findUser(int id) {
    User user = database.lookup(id);
    if (user == null) {
        throw new UserNotFoundException("No user with id " + id);
    }
    return user;
}

```

Custom checked exception

Extend `Exception` directly.

```

java

public class InvalidConfigException extends Exception {
    public InvalidConfigException(String message) {
        super(message);
    }
}

```

Anyone who calls a method throwing this must handle it or declare it.

Adding extra fields

Custom exceptions can carry more info than just a message.

```

java

public class ApiException extends RuntimeException {
    private final int statusCode;
    private final String endpoint;

    public ApiException(int statusCode, String endpoint, String message) {
        super(message);
        this.statusCode = statusCode;
        this.endpoint = endpoint;
    }

    public int getStatusCode() { return statusCode; }
    public String getEndpoint() { return endpoint; }
}

```

Now the catcher can react based on the status code.

```

java

```

```
try {
    callApi("/users");
} catch (ApiException e) {
    if (e.getStatusCode() == 404) {
        // handle not found
    }
}
```

When to make a custom exception

Make one when:

- The built-in ones do not fit
- You want callers to handle this specific case differently
- You need to add extra context (like fields above)

Do not overdo it

You do not need a separate exception for every error. If `IllegalArgumentException` works, just use it.

Should it be checked or unchecked?

Modern advice: prefer unchecked exceptions for most cases.

- Unchecked: cleaner code, lets exceptions bubble up naturally
- Checked: forces handling, but adds noise everywhere

Use checked only if the caller can reasonably recover and you want to force them to think about it. Most modern frameworks (like Spring) use unchecked exceptions everywhere.

Exception Chaining

Sometimes you catch an exception and throw a new one. Keep the original around so you do not lose information.

```
java

public User loadUser(int id) {
    try {
        return fetchFromDatabase(id);
    } catch (SQLException e) {
        throw new ServiceException("Failed to load user " + id, e);
    }
}
```

Notice the second argument to `ServiceException`. That is the "cause." It keeps the original exception so you can see what really went wrong.

Reading the cause

```
java
try {
    service.loadUser(1);
} catch (ServiceException e) {
    System.err.println("Service failed: " + e.getMessage());

    Throwable cause = e.getCause();
    if (cause != null) {
        System.err.println("Caused by: " + cause.getMessage());
    }
}
```

When you print the stack trace, you see the full chain:

```
ServiceException: Failed to load user 1
    at ...
Caused by: SQLException: connection refused
    at ...
```

Why chain exceptions?

Chaining gives you the best of both worlds. You get a high-level exception that makes sense at this layer of code, and you keep the technical details about what really broke.

This is critical for debugging in production.

Helpful NullPointerException (Java 14+)

Before Java 14, an NPE message was vague:

```
Exception in thread "main" java.lang.NullPointerException
    at com.example.MyApp.process(MyApp.java:42)
```

You had to look at line 42 and guess which thing was null.

Now Java tells you exactly what was null:

```
Exception in thread "main" java.lang.NullPointerException:
Cannot invoke "String.length()" because "user.address.street" is null
```

This is on by default since Java 15. It saves a lot of debugging time.

Best Practices

1. Be specific in catches

```
java

// Bad: catches too much
catch (Exception e) { ... }

// Better: be specific
catch (IOException e) { ... }
catch (SQLException e) { ... }
```

Catch only what you can actually handle.

2. Never swallow exceptions

```
java

// Very bad: empty catch block
try {
    riskyOp();
} catch (Exception e) {
    // nothing here
}
```

You will regret this later when something breaks and there are no logs.

```
java

// Better: log it at minimum
try {
    riskyOp();
} catch (Exception e) {
    log.error("Operation failed", e);
    throw new ServiceException("Operation failed", e);
}
```

If you really must ignore an exception, leave a comment explaining why.

3. Use try-with-resources for anything closeable

```
java
```

```
// Bad
FileReader r = new FileReader("file.txt");
try {
    // ...
} finally {
    r.close();
}

// Good
try (FileReader r = new FileReader("file.txt")) {
    // ...
}
```

4. Do not use exceptions for control flow

```
java

// Bad: using exceptions for normal logic
public boolean isNumber(String s) {
    try {
        Integer.parseInt(s);
        return true;
    } catch (NumberFormatException e) {
        return false;
    }
}

// Better
public boolean isNumber(String s) {
    return s != null && s.matches("-?\\d+");
}
```

Exceptions are slow and unclear when used for normal flow.

5. Throw early, catch late

Validate inputs at the start of a method. Let exceptions bubble up to where they can be handled meaningfully.

```
java
```

```

public void process(User user, Order order) {
    if (user == null) {
        throw new IllegalArgumentException("user cannot be null");
    }
    if (order == null) {
        throw new IllegalArgumentException("order cannot be null");
    }

    // proceed knowing inputs are good
}

```

6. Do not catch Throwable or Error

```

java

catch (Throwable t) { ... }    // bad
catch (Error e) { ... }        // bad

```

These include things like `OutOfMemoryError`. Your code usually cannot recover.

7. Document exceptions

```

java

/**
 * Loads a user by id.
 *
 * @throws UserNotFoundException if no user has the given id
 * @throws DatabaseException if the database is unreachable
 */
public User loadUser(int id) { ... }

```

Tells callers what they need to handle.

8. Wrap when crossing layers

```

java

// In data layer
catch (SQLException e) {
    throw new DataAccessException("Failed to load", e);
}

```

Do not let `SQLException` leak to your service layer or web layer. Wrap it in something more meaningful.

9. Use finally only for cleanup, not logic

```
java

// Bad
try {
    return computeValue();
} finally {
    return -1;    // overrides the return!
}
```

Putting a `return` in `finally` hides the real result. Avoid.

10. Prefer unchecked for programming errors

If something is a bug (like a null parameter or wrong state), throw an unchecked exception. The caller probably needs to fix their code, not handle the error.

Exception Handling in Streams

Stream operations do not work nicely with checked exceptions. The functional interfaces (`Function`, `Predicate`, etc.) do not declare them.

The problem

```
java

List<String> paths = List.of("a.txt", "b.txt");

paths.stream()
    .map(p -> Files.readString(Path.of(p)))    // compile error!
    .toList();
```

`Files.readString` throws `IOException`. The lambda cannot declare it.

Solution 1: Wrap inline

```
java

List<String> contents = paths.stream()
    .map(p -> {
        try {
            return Files.readString(Path.of(p));
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    })
    .toList();
```

Works, but verbose.

Solution 2: Helper method

```
java

public static String readFile(String path) {
    try {
        return Files.readString(Path.of(path));
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}

// Now the stream is clean
List<String> contents = paths.stream()
    .map(MyClass::readFile)
    .toList();
```

This is the cleaner approach.

Solution 3: Skip failures

If you want to ignore failures and continue.

```
java

List<String> contents = paths.stream()
    .map(p -> {
        try {
            return Files.readString(Path.of(p));
        } catch (IOException e) {
            return null;
        }
    })
    .filter(Objects::nonNull)
    .toList();
```

A note on libraries

Libraries like Vavr or Lombok's `@SneakyThrows` offer cleaner ways. But they add a dependency. The helper method approach is usually enough.

Putting It All Together

Real examples that combine these features.

Example 1: A typical service method

```
java

public class UserService {

    public User getUser(int id) {
        if (id <= 0) {
            throw new IllegalArgumentException("id must be positive");
        }

        try {
            return userRepository.findById(id)
                .orElseThrow(() -> new UserNotFoundException("No user " + id));
        } catch (DataAccessException e) {
            log.error("Database error fetching user {}", id, e);
            throw new ServiceException("Failed to load user", e);
        }
    }
}
```

This shows: input validation, exception chaining, layered exceptions, and logging.

Example 2: Reading a file safely

```
java

public Optional<String> readFileContent(String path) {
    try (BufferedReader reader = Files.newBufferedReader(Path.of(path))) {
        StringBuilder sb = new StringBuilder();
        String line;
        while ((line = reader.readLine()) != null) {
            sb.append(line).append("\n");
        }
        return Optional.of(sb.toString());
    } catch (IOException e) {
        log.warn("Could not read file: {}", path, e);
        return Optional.empty();
    }
}
```

Uses try-with-resources, returns Optional instead of null, logs the failure.

Example 3: API call with multiple failure modes

```
java
```

```
public Order placeOrder(OrderRequest request) {
    try {
        validateRequest(request);
        Inventory inv = inventoryService.reserve(request.items());
        Payment pay = paymentService.charge(request.payment());
        return orderRepository.save(new Order(request, inv, pay));
    } catch (InventoryException e) {
        throw new OrderFailedException("Out of stock", e);
    } catch (PaymentException e) {
        throw new OrderFailedException("Payment failed", e);
    } catch (Exception e) {
        log.error("Unexpected error placing order", e);
        throw new OrderFailedException("Unexpected error", e);
    }
}
```

Different exception types, all wrapped in a single high-level exception for the caller.

Example 4: Custom exception with extra context

```
java
```

```

public class ValidationException extends RuntimeException {
    private final String field;
    private final Object rejectedValue;

    public ValidationException(String field, Object rejectedValue, String message) {
        super(message);
        this.field = field;
        this.rejectedValue = rejectedValue;
    }

    public String getField() { return field; }
    public Object getRejectedValue() { return rejectedValue; }
}

// Throwing
if (age < 0) {
    throw new ValidationException("age", age, "Age cannot be negative");
}

// Catching
catch (ValidationException e) {
    return ResponseEntity.badRequest().body(Map.of(
        "field", e.getField(),
        "value", e.getRejectedValue(),
        "message", e.getMessage()
    ));
}

```

Useful in REST APIs where you need to return structured error responses.

Common Mistakes

Mistake 1: Catching too broad

```

java

catch (Exception e) {
    // could be anything, hard to handle properly
}

```

Catch the specific type when you can.

Mistake 2: Empty catch blocks

```
java

try {
    riskyOp();
} catch (Exception e) {
    // nothing!
}
```

At minimum, log the exception. Silence is the enemy of debugging.

Mistake 3: Catching and rethrowing without context

```
java

catch (SQLException e) {
    throw new RuntimeException(e);    // lost the message
}

// Better:
catch (SQLException e) {
    throw new ServiceException("Failed to load user " + id, e);
}
```

Mistake 4: Not using try-with-resources

```
java

// Old, error-prone
FileReader r = new FileReader("file.txt");
try {
    // ...
} finally {
    r.close();
}

// Use this instead
try (FileReader r = new FileReader("file.txt")) {
    // ...
}
```

Mistake 5: Using exceptions for control flow

```
java
```

```
// Bad
try {
    return map.get(key).toString();
} catch (NullPointerException e) {
    return "default";
}

// Better
return map.containsKey(key) ? map.get(key).toString() : "default";
```

Mistake 6: Catching NullPointerException

```
java

catch (NullPointerException e) {
    // almost always means a bug
}
```

This usually means you should have a null check, not a catch. Fix the cause.

Mistake 7: Returning null from a method that throws

```
java

public User findUser(int id) {
    try {
        return database.find(id);
    } catch (Exception e) {
        return null;    // caller has no idea something failed
    }
}
```

Either let the exception propagate or return `Optional.empty()`. Returning null silently hides the problem.

Mistake 8: Forgetting to close in finally for non-AutoCloseable

If a resource does not implement `AutoCloseable`, you still need a finally block. Make sure you do not forget.

Mistake 9: Throwing checked exceptions from common interfaces

```
java
```

```
public class MyComparator implements Comparator<String> {
    @Override
    public int compare(String a, String b) throws IOException {    // does not compile
        // ...
    }
}
```

The interface does not declare `IOException`, so you cannot add it. Wrap in unchecked.

Mistake 10: Swallowing InterruptedException

```
java
try {
    Thread.sleep(1000);
} catch (InterruptedException e) {
    // ignored, BAD
}
```

Always at minimum re-set the interrupt flag:

```
java
catch (InterruptedException e) {
    Thread.currentThread().interrupt();
    // optionally rethrow as RuntimeException
}
```

Quick Cheat Sheet

You want to...	Use
Catch an exception	<code>try { ... } catch (Type e) { ... }</code>
Run cleanup code always	<code>finally</code> block
Auto-close a resource	try-with-resources <code>try (Resource r = ...)</code>
Throw an exception	<code>throw new SomeException(...)</code>
Declare a checked exception	<code>throws SomeException</code> in method signature
Catch multiple types the same way	<code>catch (TypeA TypeB e)</code>

You want to...	Use
Wrap an exception with new context	<code>throw new MyEx("...", originalException)</code>
Make an unchecked exception	extend <code>RuntimeException</code>
Make a checked exception	extend <code>Exception</code>
Get the original cause	<code>e.getCause()</code>
Get the message	<code>e.getMessage()</code>
Print the stack trace	<code>e.printStackTrace()</code> (or use a logger)

Quick Reference: Common Exceptions

Exception	Means
<code>NullPointerException</code>	Used something that was null
<code>IllegalArgumentException</code>	Bad argument passed to method
<code>IllegalStateException</code>	Object is in wrong state for this operation
<code>IndexOutOfBoundsException</code>	Bad index for array, list, or string
<code>ArrayIndexOutOfBoundsException</code>	Bad array index specifically
<code>ClassCastException</code>	Wrong cast
<code>NumberFormatException</code>	Bad string for <code>parseInt</code> , etc.
<code>ConcurrentModificationException</code>	Changed a collection while iterating
<code>ArithmeticException</code>	Math problem like divide by zero
<code>UnsupportedOperationException</code>	Method not supported on this object
<code>IOException</code> (checked)	I/O failure
<code>SQLException</code> (checked)	Database failure
<code>InterruptedException</code> (checked)	Thread was interrupted

Final Tips

1. Catch specific exceptions, not `Exception` or `Throwable`. Be intentional about what you handle.
2. Always use try-with-resources for files, streams, and connections. Forget about manual `finally` cleanup.
3. Never have an empty catch block. At minimum, log the exception.
4. Throw early, catch late. Validate inputs at the start of methods. Let exceptions bubble up to where they can be handled meaningfully.
5. Wrap exceptions when crossing layer boundaries. Do not let `SQLException` leak into your web controller.
6. Prefer unchecked exceptions in modern Java. Most frameworks (Spring, Hibernate, etc.) use unchecked exceptions everywhere.
7. Add helpful messages to exceptions. "Failed to load user 42" is much better than "Failed".
8. Do not use exceptions for normal control flow. They are slow and confusing.
9. When chaining exceptions, always include the original as the cause. Do not throw away information.
10. Read stack traces carefully. The first line of "Caused by:" usually tells you the real problem.

Exception handling is one of those skills that separates beginner code from production-quality code. The patterns are simple, but using them well takes practice.

When you are ready, good next topics are:

- Java Collections deep dive (when to use which collection)
- Multithreading and Concurrency (synchronized, locks, atomic classes)
- Common design patterns for interviews
- JVM internals and memory management

Happy coding!